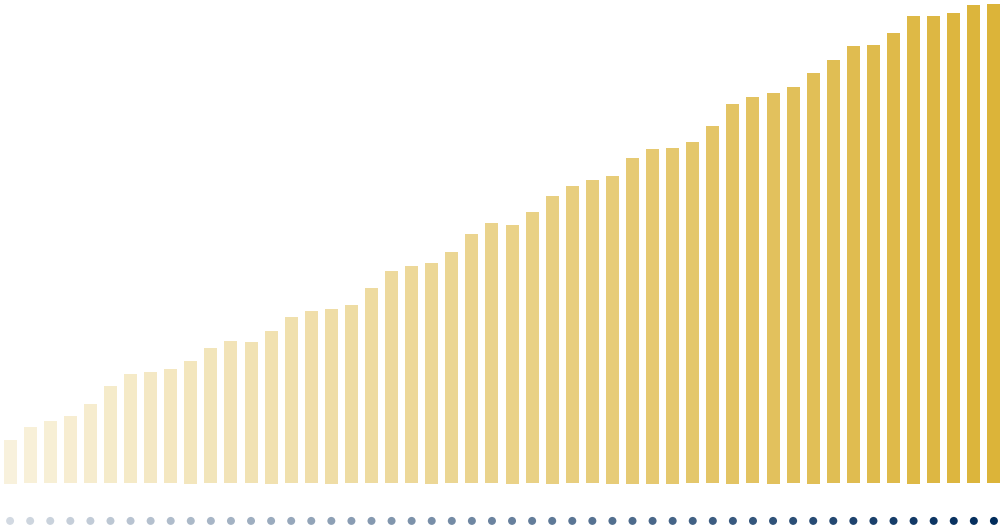


UNIT 03 · CHAPTER 4

Backpropagation Through Time

CSE 30124 · Introduction to Artificial Intelligence

William Theisen · University of Notre Dame

Feed-Forward Networks

(old exam question)

```python
import torch
import torch.nn as nn
class FFN(nn.Module):
    def __init__(self):
        super(FFN, self).__init__()
        self.F1 = nn.Flatten()
        self.L1 = nn.Linear(28*28, 512)
        self.R1 = nn.Sigmoid()
        self.L2 = nn.Linear(512, 256)
        self.R2 = nn.Sigmoid()
        self.L3 = nn.Linear(256, 10)
    def forward(self, x):
        x = self.F1(x)
        x = self.L1(x)
        x = self.R1(x)
        x = self.L2(x)
        x = self.R2(x)
        x = self.L3(x)
        return x
```

a.) Draw the computation graph for the network.

Feed-Forward Networks

(old exam question)

Note: You can assume `softmax` and `cross entropy loss` are used after the final layer, the two yielding a combined derivative of: $\hat{y} - y$

b.) Give the loss as a single nested equation.

$\mathcal{L} =$

c.) Give the derivative chain for calculating the gradient of the bias in the first layer.

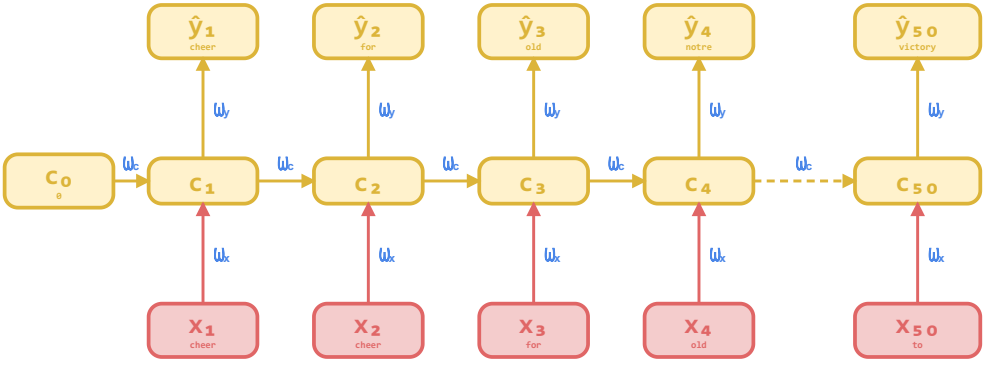
$$\frac{\partial \mathcal{L}}{\partial b_1} =$$

d.) Write the actual equation you'd use to calculate the gradient for the bias in the first layer.

$$\frac{\partial \mathcal{L}}{\partial b_1} =$$

Where We Left Off

Last time our RNN's weights were _____. To learn them we unroll it over the whole chorus: **51** words, **41** different, a hidden state of **16** numbers, and at every word _____



Loss Over a Sequence

Unrolled, an RNN is just

_____

that reuses its weights in every layer

Every copy makes a prediction, so every copy gets a loss:

Knowing nothing (`1` in `41` for every word) costs

_____,

our random RNN scores `3.719`

Computing the Loss

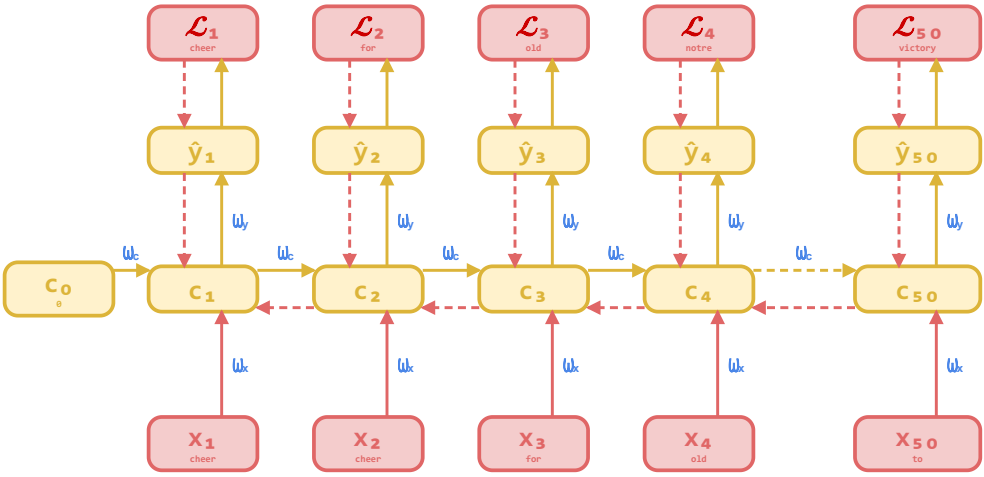
```
import numpy as np
import torch
song = """cheer cheer for old notre dame wake up the
echoes cheering her name send a volley cheer on
high shake down the thunder from the sky what
though the odds be great or small old notre dame
will win over all while her loyal sons and
daughters march on to victory"""
words = song.split()
vocab = list(dict.fromkeys(words))  # first appearance
seq = [vocab.index(w) for w in words]
x, y = seq[:-1], seq[1:]       # each word, the next
V, H, T = len(vocab), 16, len(x)
rng = np.random.default_rng(0)  # same start as before
f = lambda d, n: torch.tensor(rng.normal(0, d, n)).float()
Wx = f(V ** -0.5, (H, V))
Wc = f(H ** -0.5, (H, H))
Wy = f(H ** -0.5, (V, H))
bc, by = torch.zeros(H), torch.zeros(V)
c, loss = torch.zeros(H), 0
for t in range(T):
    c = torch.tanh(Wx[:, x[t]] + c @ Wc.T + bc)
    o = c @ Wy.T + by
    p = torch.softmax(o, dim=0)
    loss -= torch.log(p[y[t]])
print(V, T, round(loss.item() / T, 3),
      round(np.log(V), 3))
```

OUTPUT

```
41 50 3.719 3.714
```

Backpropagation Through Time

Backpropagation through time (BPTT) is just _____, from the last word back to the first



Each hidden state gets its gradient from _____ (z_t is what goes into the tanh):

Gradients of Shared Weights

Every copy uses the same weights, so each weight's gradient is _____:

Training on the Victory March

`1,000` steps of BPTT by hand (α = `0.5`): the loss goes from `3.719` to _____, and the RNN gets _____ next words right, both "dame"s included ("wake", then "will")

Gradient checking: nudge one weight of W_c up and down, and the slope should match

---

(next slide, with PyTorch doing the BPTT)

BPTT in PyTorch

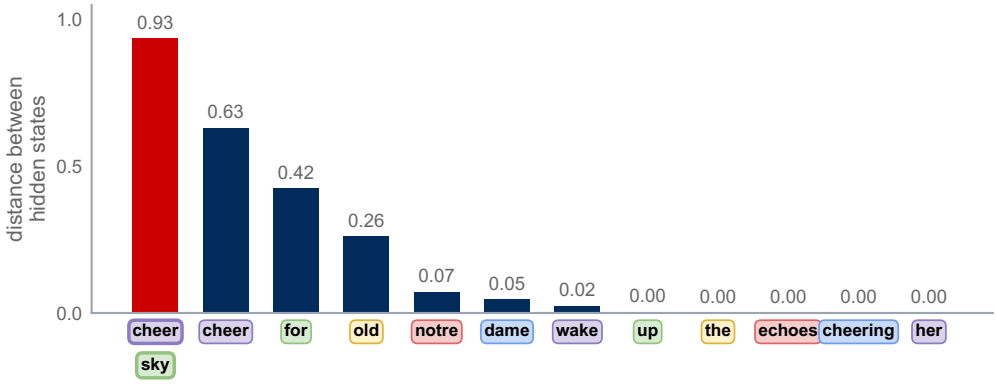
```
import numpy as np
import torch
song = """cheer cheer for old notre dame wake up the
echoes cheering her name send a volley cheer on high
shake down the thunder from the sky what though the
odds be great or small old notre dame will win over all
while her loyal sons and daughters march on to victory"""
vocab = list(dict.fromkeys(song.split()))
s = [vocab.index(w) for w in song.split()]
V, H, T = len(vocab), 16, len(s) - 1
rng = np.random.default_rng(0)       # the same start
f = lambda d, n: torch.tensor(rng.normal(0, d, n))
Wx, Wc = f(V ** -0.5, (H, V)), f(H ** -0.5, (H, H))
Wy = f(H ** -0.5, (V, H))
bc, by = torch.zeros(H).double(), torch.zeros(V).double()
def loss_of(Wc):
    c, loss = torch.zeros(H).double(), 0
    for t in range(T):
        c = torch.tanh(Wx[:, s[t]] + c @ Wc.T + bc)
        p = torch.softmax(c @ Wy.T + by, dim=0)
        loss = loss - torch.log(p[s[t + 1]])
    return loss / T
Wc.requires_grad_()
loss_of(Wc).backward()                # BPTT, for us
e = torch.zeros(H, H).double()
e[2, 3] = 1e-6                         # one weight
nudged = (loss_of(Wc + e) - loss_of(Wc - e)) / 2e-6
print(round(Wc.grad[2, 3].item(), 8),
      round(nudged.item(), 8))
```

OUTPUT

```
-0.01508333 -0.01508333
```

Long-Term Dependencies

In last chapter's RNN (made-up weights), change the first word from **cheer** to **sky** and measure how far apart the hidden states are after each word:



By the ninth word it's `0.001`: the **RNN** _____ how the song started. A word that depends on one far back is a **long-term dependency**. One fix:

_____

Vanishing/Exploding Gradients

RNNs are particularly **vulnerable to the vanishing gradient problem** because we have to **compute the gradients of the loss**

Like the sigmoids in the review's **d.)**, every word we walk back multiplies the gradient by a tanh derivative D_t and by W_c:

one tanh derivative and one weight matrix per word

$$\frac{\partial \mathcal{L}_{50}}{\partial c_k} = \frac{\partial \mathcal{L}_{50}}{\partial c_{50}} D_{50} W_c \, D_{49} W_c \cdots D_{k+1} W_c$$

Vanishing Gradient

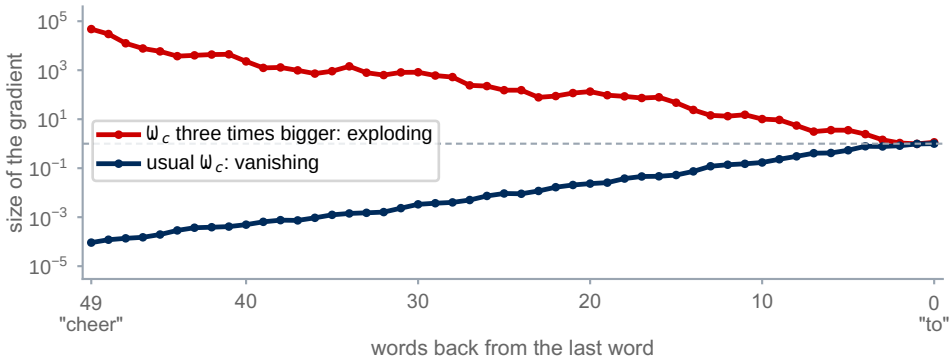
Since you are multiplying increasingly _____ values with each other during backprop, the gradient eventually trends to _, meaning **updates will stop happening**.



Exploding Gradient

Almost the opposite of the vanishing gradient, the exploding gradient is when the gradients become

Vanishing/Exploding Gradients



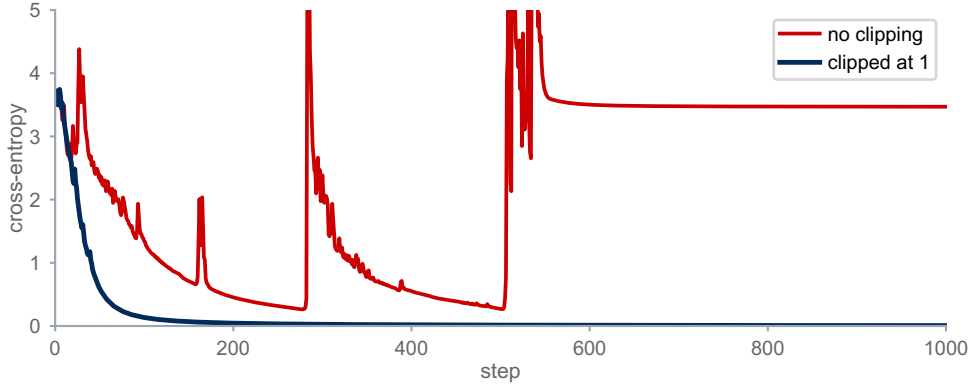
Gradient Clipping

An _____ gradient has a simple fix

Clip, or put a ceiling on, the size of the gradient we'll accept. Vanishing gradients are harder: they need a new kind of cell.

[4.5] if the gradient's norm is over the ceiling τ, scale it down to τ

$$\mathbf{g} \leftarrow \tau \frac{\mathbf{g}}{\|\mathbf{g}\|} \qquad \text{if } \|\mathbf{g}\| > \tau$$

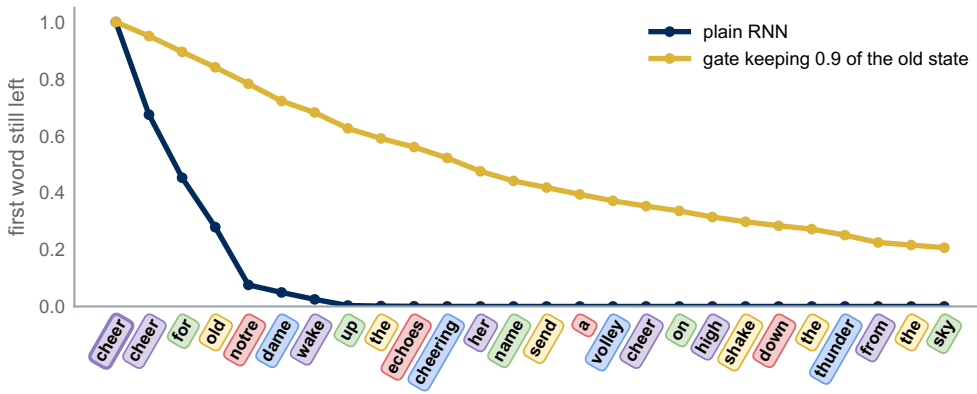


Gates

Let each step choose how much of the _____ to keep:

z is between 0 and 1

$$c_t = z_t \odot c_{t-1} + (1 - z_t) \odot \tilde{c}_t$$



Gradients Through a Gate

The gate keeps part of the old hidden state:

The straight path back has _____: a gate of `0.9` leaves $0.9^{49} = 0.0057$ after `49` words, about `60` times what reached the first word in our RNN

LSTMs and GRUs

A real gate is _____, so the network _____

- **GRU** (2014): _____
- **LSTM** (1997): _____

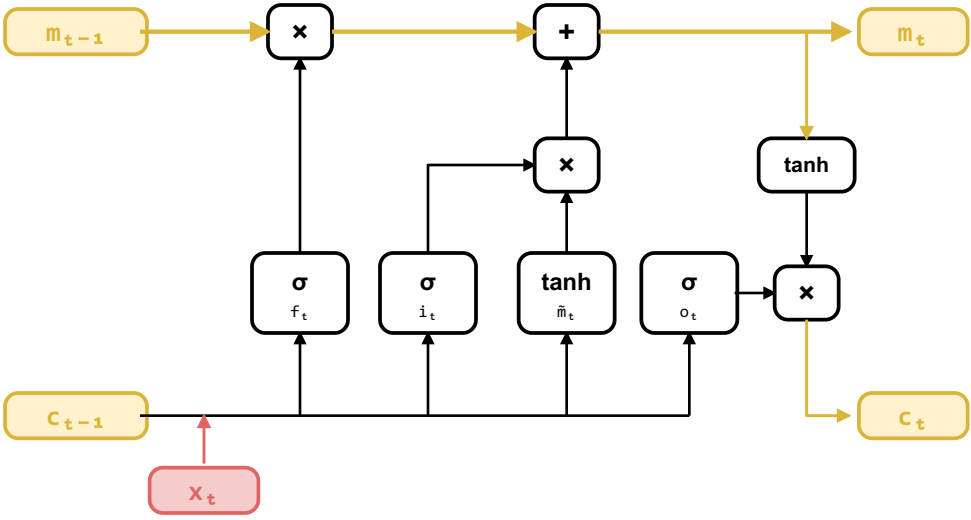
```
from torch import nn
for cell in [nn.RNN(2, 2), nn.GRU(2, 2), nn.LSTM(2, 2)]:
    n = sum(p.numel() for p in cell.parameters())
    print(type(cell).__name__, n)
```

OUTPUT

```
RNN 12
GRU 36
LSTM 48
```

Long Short-Term Memory units - LSTMs (1997)

LSTMs introduce the idea of _____ to help solve the _____ gradient problem.



Breaking Down LSTMs

An LSTM on the Victory March

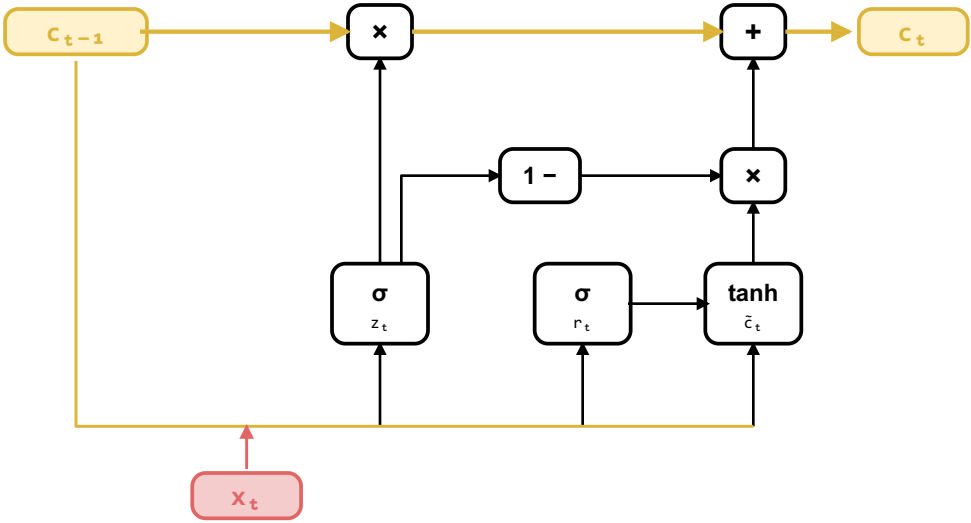
word		f	i	$\tilde{m}$	o	m	c
4	old	0.91	0.94	−0.97	1.00	−2.25	−0.97
5	notre	0.93	0.94	0.53	1.00	−1.59	−0.92
6	dame	0.79	0.95	0.92	1.00	−0.38	−0.36
35	old	0.97	0.89	−0.96	1.00	−8.18	−1.00
36	notre	0.97	0.89	0.31	1.00	−7.66	−1.00
37	dame	0.91	0.88	0.77	1.00	−6.33	−1.00

The two "dame"s get the same input, the cell state tells them apart:

Gated Recurrent Units - GRUs (2014)

GRUs are a **simpler version of LSTMs** that

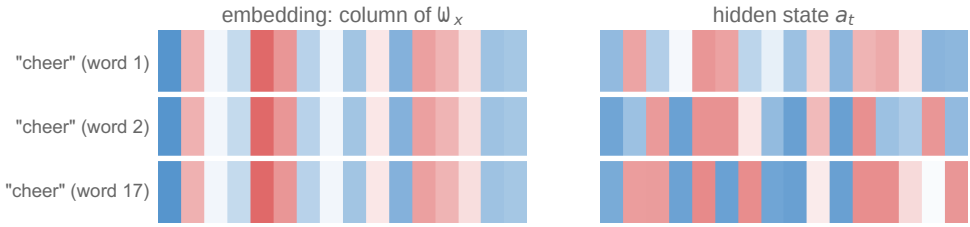
to simplify the gate structure



Static Embeddings

For this lecture, we **sadly only receive static embeddings**, or embeddings that are

---



Concept Cells

Quiroga et al. (2005) recorded single neurons in the brains of epilepsy patients (who already had electrodes in for surgery) while showing them pictures.

One neuron fired for **photos of Jennifer Aniston**, and almost nothing else. Another fired for different photos of Halle Berry, a drawing of her, and **her name written out**.

Like a **word embedding**, the brain seems to keep **one representation of a** _____, whichever form (picture, drawing, word) it arrives in.

The Bottleneck

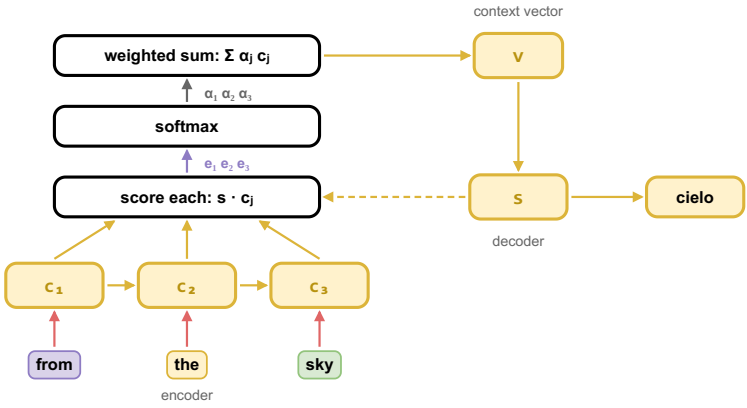
An encoder-decoder has to fit the whole sentence into _____ (last chapter's RNN: `2` numbers for the whole song)

Cho et al. (`2014`): translations got **steadily worse as sentences got longer**

But the encoder made a hidden state _____. Why throw them away?

Attention

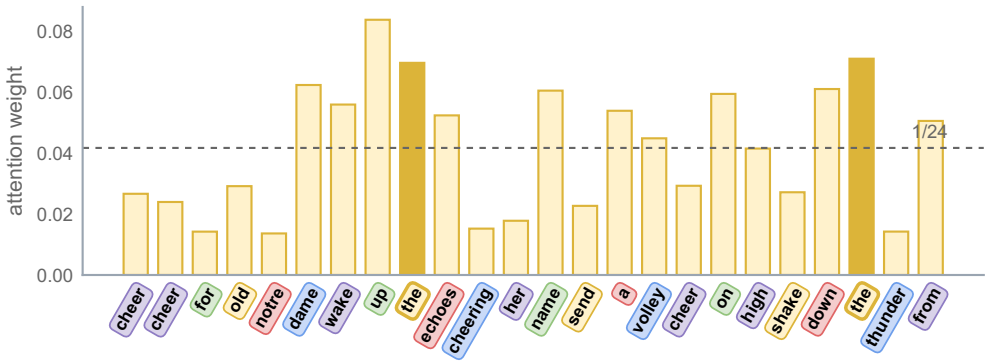
Bahdanau, Cho and Bengio (2014): before each word it writes, the decoder



Every word written gets

Attention Weights

Last chapter's network at the last **the**, looking back at the 24 words before it:



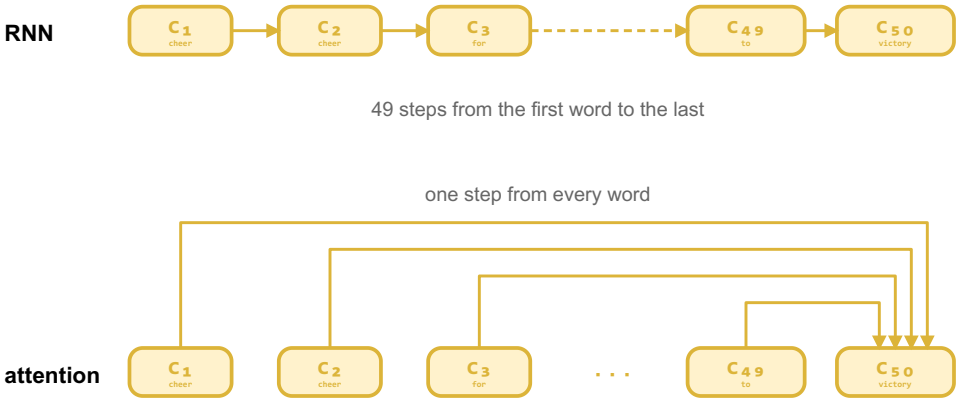
Most weight on **up** and the other **the**s:

_____.

Next chapter: attention with learned weights

Attention

Some parts of our **context** are **more important than others**! **Attention** is a mechanism through which we can _____



RNN Comeback? (Feb. 2026)

Memory Caching (Behrouz and others, Google Research): an RNN that saves its hidden state at the end of every segment, so later words can

---

